Depth First Search (DFS) Lab Session

Problem Statement

Write a program to store a town's road network as a graph and explore it using Depth First Search (DFS).

The road network of a small town is recorded as a graph: each junction is a vertex, and each stretch of road joining two junctions is an edge. Roads can be driven in either direction, so the graph is undirected.

We would like to walk the network systematically, starting from a junction and going as deep as we can before turning back, and we would like to know how many separate pieces the network falls into. Two junctions are in the same piece exactly when you can drive from one to the other.

Store the network as an adjacency list and run a depth first search (DFS) on it.

Input Format

The graph is read from standard input, and the answer is written to standard output. Your program is run as `./dfs < input01.txt > output01.txt`, so do not print a prompt or ask for a file name. Print nothing except the two output lines described below. The skeleton also accepts a file name as a command line argument (`./dfs input01.txt`), which is convenient while testing, but the graded runs use standard input.

- The first line contains a single integer n, the number of vertices. The vertices are numbered $1, 2, \ldots, n$.
- Then follow n lines, one per vertex. Line i describes vertex i: it begins with the degree d_i of vertex i, followed by the d_i neighbours of vertex i, all separated by single spaces. A vertex of degree 0 gives a line containing just `0`.

The graph is undirected and the input is consistent: vertex v appears on line u if and only if vertex u appears on line v. There are no self loops and no repeated neighbours.

Output Format

Print exactly two lines:

1. The first line is the order in which DFS visits the vertices: the n vertex numbers separated by single spaces, ending with a newline.
2. The second line is the number of connected components of the graph, ending with a newline.

How the Search Must Run

Start the search at vertex 1.

When the search from a vertex can go no further and every vertex reachable from the start has been visited, restart it at the smallest numbered vertex that has not yet been visited. Each restart is a new connected component. Repeat until every vertex has been visited.

At a vertex u, consider the neighbours of u in the order in which they appear on line u of the input, and recurse into the first one that has not been visited yet. This is what makes the answer unique, so your adjacency list must preserve the order of each line: a neighbour that is read later must end up later in the list. Inserting each new neighbour at the front of the list reverses the order and gives a different, wrong answer.

Constraints

- $1 \leq n \leq 1000$, and $0 \leq d_i \leq n - 1$.
- The number of edges is at most 10^4.
- The graph may be disconnected, and may have no edges at all.

Implementation Rules

- The graph must be stored as an adjacency list: an array indexed by vertex, each entry a linked list of that vertex's neighbours. Do not use an adjacency matrix.
- The search must be a depth first search as described above. Do not compute the components by any other method. The count must come out of the same traversal.
- You are given a skeleton, `graph_skeleton.c`, which already reads the graph and prints the answer. Fill in the two places marked `BLANK 1` and `BLANK 2` and change nothing else.

Examples

Example 1

INPUT:

```
6
2 3 2
2 1 4
2 1 4
3 2 3 5
2 4 6
1 5
```

OUTPUT:

```
1 3 4 2 5 6
1
```

Explanation: Line 1 reads `2 3 2`: vertex 1 has degree 2, and its neighbours are listed as 3 first and 2 second, so the search goes to 3 before it goes to 2.

The search goes $1 \to 3 \to 4 \to 2$, backtracks from 2 (both of its neighbours are already visited) and from 4 to reach 5, then 6. The whole graph is one component, so the second line is 1.

Example 2 (Multiple Components)

INPUT:

```
7
1 3
1 4
2 1 5
2 2 6
2 3 7
1 4
1 5
```

OUTPUT:

```
1 3 5 7 2 4 6
2
```

Explanation: This graph has seven vertices and falls into two pieces: the odd numbered vertices form the path $1-3-5-7$, and the even numbered ones form the path $2-4-6$. There is no edge between the two pieces.

Starting at vertex 1, the search reaches 3, then 5, then 7, and can go no further; at that point vertices 2, 4, 6 have not been visited. The smallest of them is 2, so the search restarts there and walks $2 \to 4 \to 6$. Two restarts were needed in all, one at vertex 1 and one at vertex 2, so the graph has two connected components and the second output line is 2. Notice that the visit order is not $1, 2, \ldots, n$: the first component is finished completely before the second one is begun.

Instructions & Academic Honesty

Attempt this assignment on your own. Do not refer to external sources while writing your solution. This includes code found on websites, code from your classmates or seniors, and large language models such as ChatGPT, Claude, Gemini or Copilot. You have been given a skeleton in which all of the reading, printing and bookkeeping is already written, and only a small part of the code is left for you to supply; working that part out yourself is the entire point of the exercise, and it is also what the examinations will test. You are free to use your lecture notes and the prescribed textbook. Any submission that appears to have been copied will be handled according to the course policy on academic honesty.

Remarks

- See the public test cases to understand the I/O format. Each one is a pair of text files, `inputNN.txt` and `outputNN.txt`.
- **Debugging check:** The first output line must contain every vertex from 1 to n exactly once. If a vertex is missing or repeated, the visited array is not being used correctly.
- **Important:** Remove all debug output before submitting your code! Your output is compared exactly against the expected output, so a single extra line, prompt or summary will fail every test case.